

Key Technologies

Redis

Learn about how you can use Redis to solve a large number of problems in System Design.



Watch Video Walkthrough

Watch the author walk through the problem step-by-step



Watch
Now

System designs can involve a dizzying array of different technologies, concepts and patterns, but one technology (arguably) stands above the rest in terms of its *versatility*: Redis. This versatility is important in an interview setting because it allows you to go deep. Instead of learning about dozens of different technologies, you can learn a few useful ones and learn them deeply, which magnifies the chances that you're able to get to the level your interviewer is expecting.

Beyond versatility, Redis is great for its *simplicity*. Redis has a ton of features which resemble data structures you're probably used to from coding (hashes, sets, sorted sets, streams, etc) and which, given a few basics, are easy to reason about how they behave in a distributed system. While many databases involve a lot of magic (optimizers, query planners, etc), with only minor exceptions Redis has remained quite simple and good at what it does best: executing simple operations **fast**.

Ok, Redis is versatile, simple, and useful for system design interviews. Let's learn how it works.

Redis Basics

Redis is a self-described "data structure store" written in C. It's in-memory 🤪 and single threaded 🤖 making it very fast and easy to reason about.



One important reason you might not want to use Redis is because you need **durability**. While there are some

reasonable strategies for (using Redis' Append-Only File AOF) to *minimize* data loss, you don't get the same guarantees you might get from e.g. a relational database about commits being written to disk. This is an intentional tradeoff made by the Redis team in favor of speed, but alternative implementations (e.g. AWS' MemoryDB) will compromise a bit on speed to give you disk-based durability. If you need it, it's there!

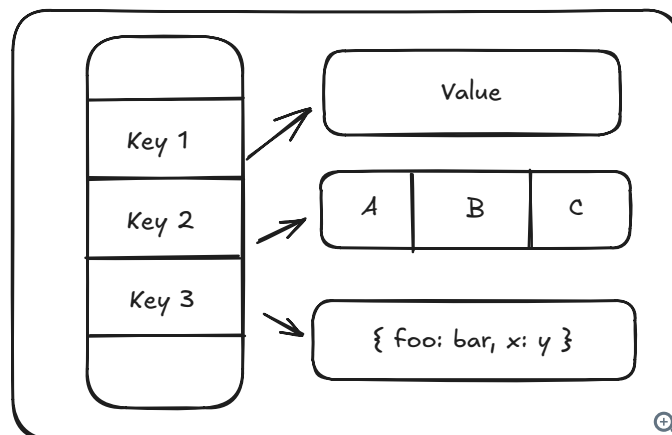
Some of the most fundamental data structures supported by Redis:

- Strings
- Hashes (objects/dictionaries)
- Lists
- Sets
- Sorted Sets (Priority Queues)
- Bloom Filters (probabilistic set membership; allows false positives)
- Geospatial Indexes
- Time Series

In addition to simple data structures, Redis also supports different communication patterns like Pub/Sub and Streams, partially standing in for more complex setups like Apache Kafka or AWS SNS (Simple Notification Service) / SQS (Simple Queue Service).

The core structure underneath Redis is a key-value store. Keys are strings while values which can be any of the data structures supported by Redis: binary data and strings, sets, lists, hashes, sorted sets, etc. All objects in Redis have a key.

Redis



Redis logical model

The choice of keys is important as these keys might be stored in separate nodes based on your **infrastructure configuration**. Effectively, the way you organize the keys will be the way you organize your data and scale your Redis cluster.

Commands

Redis' wire protocol is a custom query language comprised of simple strings which are used for all functionality of Redis. The CLI is really simple, you can literally connect to a Redis instance and run these commands from the CLI.

```
SET foo 1
GET foo      # Returns 1
INCR foo     # Returns 2
XADD mystream * name Sara surname OConnor # Adds a
```

The **full set of commands** is surprisingly readable, when grouped by data structure. As an example, Redis' Sets support simple operations like adding an element to the set (**SADD**), getting the number of elements or cardinality (**SCARD** , the count of set members), listing those elements (**SMEMBERS**) and checking existence (**SISMEMBER**) - close analogs to what you would have with a Set in any general purpose programming language.

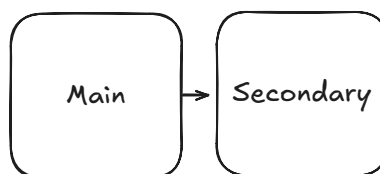
Infrastructure Configurations

Redis can run as a single node, with a high availability (HA) replica, or as a cluster. When operating as a cluster, Redis clients cache a set of "hash slots" which map keys to a specific node. This way clients can directly connect to the node which contains the data they are requesting.

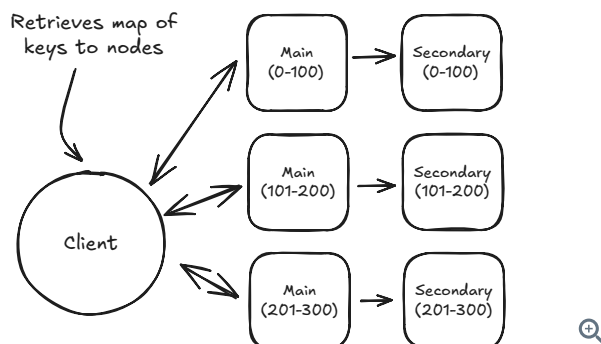
Single-Node



Replicated



Cluster



Redis infrastructure configurations



Think of hash slots like a phone book: the client keeps a local map from slots to nodes. If a slot moves during rebalancing or failover, the server replies with `MOVED` and the client refreshes its map (e.g. via `CLUSTER SHARDS`).

Each node maintains some awareness of other nodes via a gossip protocol so, in limited instances, if you request a key from the wrong node you can be redirected to the correct node. But Redis' emphasis is on performance so hitting the correct endpoint first is a priority.

Compared to most databases, Redis clusters are surprisingly basic (and thus, have some pretty severe limitations on what they can do). Rather than solving scalability problems for you, Redis can be thought of as providing you some basic primitives on which you can solve them. As an example, with few exceptions, Redis expects all the data for a given request to be on a single node!

Choosing how to structure your keys is how you scale Redis.

Performance

Redis is really, really fast. Redis can handle $O(100k)$ writes per second and read latency is often in the microsecond range. This scale makes some anti-patterns for other

database systems actually feasible with Redis. As an example, firing off 100 SQL requests to generate a list of items with a SQL database is a terrible idea, you're better off writing a SQL query which returns all the data you need in one request. On the other hand, the overhead for doing the same with Redis is rather low - while it'd be great to avoid it if you can, it's doable.

This is completely a function of the in-memory nature of Redis. It's not a good fit for every use case, but it's a great fit for many.

Capabilities

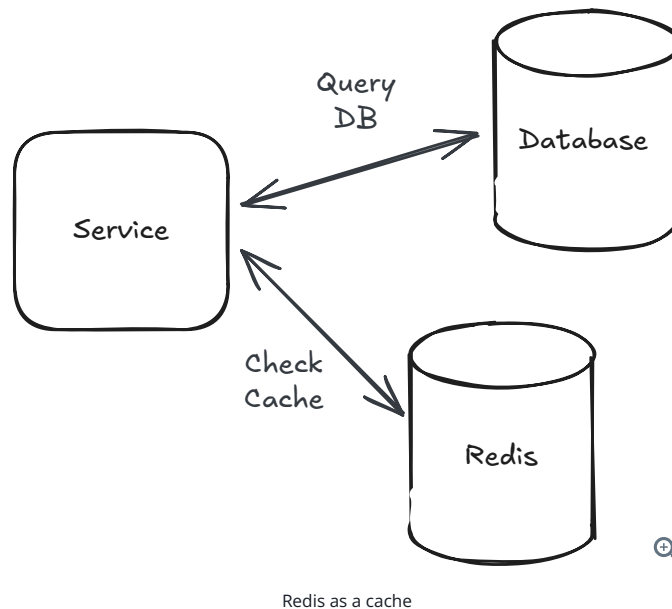
Redis as a Cache

The most common deployment scenario of Redis is as a cache. In this case, the root keys and values of Redis map to the keys and values in our cache. Redis can distribute this hash map trivially across all the nodes of our cluster enabling us to scale without much fuss - if we need more capacity we simply add nodes to the cluster.

For example, you might cache a product under key `product:123` with the value stored as a JSON blob or a Redis Hash containing fields like `name`, `price`, and `inventoryCount`.

When using Redis as a cache, you'll often employ a time to live (TTL) on each key. Redis guarantees you'll never read the value of a key after the TTL has expired and the TTL is used to decide which items to evict from the server - keeping the cache size manageable even in the case where you're trying to cache more items than memory allows.

Using Redis in this fashion doesn't solve one of the more important problems caches face: the "hot key" problem, though Redis is not unique in this respect vs alternatives like Memcached or other highly scaled databases like DynamoDB.



Redis as a Distributed Lock

Another common use of Redis in system design settings is as a distributed lock. Occasionally we have data in our system and we need to maintain consistency during updates (e.g. the very common [Design Ticketmaster](#) system design question), or we need to make sure multiple people aren't performing an action at the same time (e.g. [Design Uber](#)).



Most databases (including Redis) will offer *some* consistency guarantees. If your core database can provide consistency, don't rely on a distributed lock which may introduce extra complexity and issues. Your interviewer will likely ask you to think through the edge cases in order to make sure you really understand the concept.

A very simple distributed lock with a timeout might use the atomic increment (`INCR`) with a TTL. This is basically a shared counter. When we want to try to acquire the lock, we run `INCR` . If the response is 1 (i.e. we were the first person to try to grab the lock, so we own it!), we proceed. If the response is > 1 (i.e. someone else beat us and has the lock), we wait and retry again later. When we're done with the lock, we can `DEL` the key so that other processes can make use of it.

More sophisticated locks in Redis can use the [Redlock algorithm](#) together with fencing tokens if you want an airtight solution.

Redis for Leaderboards

Redis' sorted sets maintain ordered data which can be queried in log time which make them appropriate for leaderboard applications. The high write throughput and low read latency make this especially useful for scaled applications where something like a SQL DB will start to struggle.

In Post Search we have a need to find the posts which contain a given keyword (e.g. "tiger") which have the most likes (e.g. "Tiger Woods made an appearance..." @ 500 likes).

We can use Redis' sorted sets to maintain a list of the top liked posts for a given keyword. Periodically, we can remove low-ranked posts to save space.

```
ZADD tiger_posts 500 "SomeId1" # Add the Tiger Woods  
ZADD tiger_posts 1 "SomeId2" # Add some tweet about Tiger  
ZREMRANGEBYRANK tiger_posts 0 -6 # Remove all but top 6
```

Redis for Rate Limiting

As a data structure server, implementing a wide variety of rate limiting algorithms is possible. A common algorithm is a fixed-window rate limiter where we guarantee that the number of requests does not exceed `N` over some fixed window of time `W`.

Implementation of this in Redis is simple. When a request comes in, we increment (`INCR`) the key for our rate limiter and check the response. If the response is greater than `N`, we wait. If it's less than `N`, we can proceed. We call `EXPIRE` on our key so that after time period `W`, the value is reset.

If you need a sliding window, you can store timestamps in a Sorted Set per key and remove old entries before counting; run the check in Lua to keep it atomic.

Redis for Proximity Search

Redis natively supports geospatial indexes with commands like `GEOADD` and `GEOSEARCH`. The basic commands are simple:

```
GEOADD key longitude latitude member # Adds "member"
```

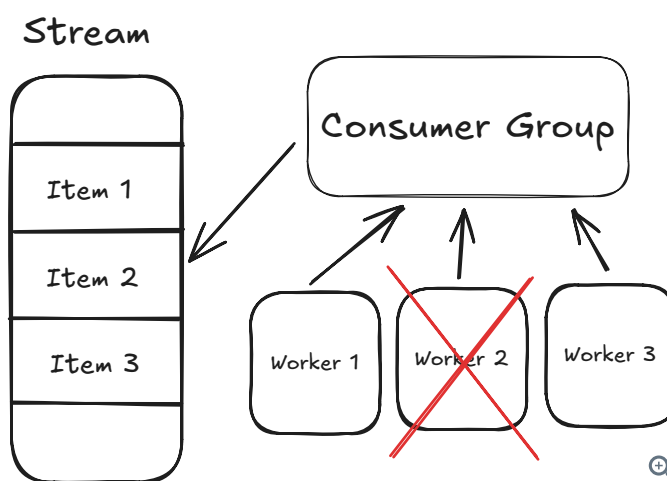
```
GEOSEARCH key FROMLONLAT longitude latitude BYRAD
```

The search command, in this instance, runs in $O(N+\log(M))$ time where N is the number of elements in the radius and M is the number of items inside the shape.

Why do we have both N and M ? Redis' geospatial commands use geohashes under the hood to index the data. These geohashes allow us to grab candidates in grid-aligned bounding boxes. But these boxes are square and imprecise. A second pass takes the candidates and filters them to only include items that are within the exact radius.

Redis for Event Sourcing

Redis' streams are append-only logs similar to Kafka's topics. The basic idea behind Redis streams is that we want to durably add items to a log and then have a distributed mechanism for consuming items from these logs. Redis solves this problem with streams (managed with commands like `XADD`) and consumer groups (commands like `XREADGROUP` and `XCLAIM`).



Redis streams and consumer groups

A simple example is a work queue. We want to add items to the queue and have them processed. At any point in time one of our workers might fail, and in these instances we'd like to re-process them once the failure is detected. With Redis streams we add items onto the queue with commands like `XADD` and have a single consumer group attached to the stream for our workers. This consumer group is maintaining a reference to the items processed via the stream and, in the case a worker fails, provides a way

for a new worker to claim (`XCLAIM`) and restart processing that message.

Redis for Pub/Sub

Redis natively supports a publish/subscribe (Pub/Sub) messaging pattern, allowing messages to be broadcast to multiple subscribers in real time. This is useful for building chat systems, real-time notifications, or any scenario where you want to decouple message producers from consumers (more discussion on this in our [Realtime Updates](#) pattern).



People frequently call out limitations of Redis Pub/Sub that are no longer true, e.g. Redis pub/sub is now sharded which enables scalability which was not possible in previous versions!

The basic commands are straightforward:

```
SPUBLISH channel message    # Sends a message to a channel
SSUBSCRIBE channel          # Listens for messages on a channel
```

When a client subscribes to a channel, it will receive any messages published to that channel as long as the connection remains open. This makes Pub/Sub great for ephemeral, real-time communication, but it's important to note that messages are not persisted—if a subscriber is offline when a message is published, it will miss that message entirely.

Pub/Sub clients use a single connection to each node in the cluster (rather than a connection per channel). Generally speaking, this means that in most cases you'll use a number of connections equal to the number of nodes in your cluster. It also means that you don't need millions of connections even if you have millions of channels!



Redis Pub/Sub is simple and fast, but not durable. The delivery of messages is "at most once" which means that if a subscriber is offline when a message is published, it will miss that message entirely. If you need message persistence, delivery guarantees, or the ability to replay missed messages, consider using Redis Streams or a dedicated message broker like Kafka or RabbitMQ.

Pub/Sub is a great fit for interview scenarios where you need to demonstrate real-time communication patterns, but be ready to discuss its limitations and when you might need a more robust solution.

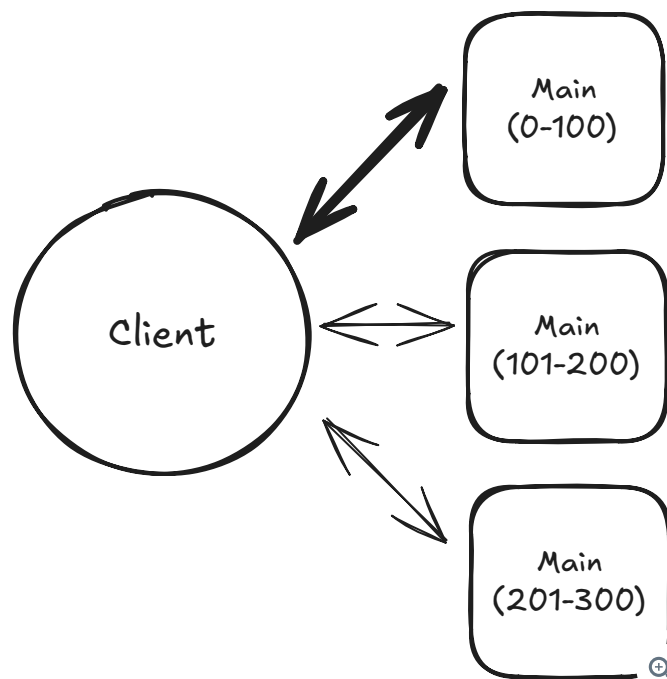


Need offline delivery or durable fan-out? Redis Streams are a good option or you can pair Pub/Sub with a queue (e.g., SNS→SQS, Kafka) or outbox pattern (i.e. write the messages to a database) so consumers can catch up later.

Shortcomings and Remediations

Hot Key Issues

If our load is not evenly distributed across the keys in our Redis cluster, we can run into a problem known as the "hot key" issue. To illustrate it, let's pretend we're using Redis to cache the details of items in our ecommerce store. We have lots of items so we scale our cluster to 100 nodes and our items are evenly spread across them. So far, so good. Now imagine one day we have a surge of interest for *one particular item*, so much that the volume for this item matches the volume for the rest of the items.



Hot key issue

Now the load on *one server* is dramatically higher than the

rest of the servers. Unless we were severely overprovisioned (i.e. we were only using a small % of the existing CPU on each node), this server is now going to start failing.

There are lots of potential solutions for this, all with tradeoffs.

- We can add an in-memory cache in our clients so they aren't making so many requests to Redis for the same data.
- We can store the same data in multiple keys and randomize the requests so they are spread across the cluster.
- We can add read replica instances and dynamically scale these with load.

For an interview setting, the important thing is that you recognize potential hot key issues (+) and that you proactively design remediations (++).

Summary

Redis is a powerful, versatile, and simple tool you can use in system design interviews. Because Redis' capabilities are based on simple data structures, reasoning through the scaling implications of your decisions is straightforward: allowing you to go deep with your interviewer without needing to know a lot of details about Redis internals.